<Optima Project 1 Solution>

Automotive Engineering 2019048440 CHOI YUN SEOK

Problem 1

1-1. Show that the functions are convex.

$$f(x_1, x_2) = 1 + 2x_1 + 3(x_1^2 + x_2^2) + 4x_1x_2$$

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^\top \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 16 & 23 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \pi^2$$

$$f(x, y) = 3(x^2 + y^2) + 4xy + 5x + 6y + 7$$

* Convex Condition

⇒ Hessian Matrix H should be positive semi-definite ($H \geq 0$)

⇒ 즉, 모든 고유값이 음수가 아니면 됨.

(1) $f(x_1, x_2) = 1 + 2x_1 + 3(x_1^2 + x_2)^2 + 4x_1x_2$

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x^2} \end{bmatrix}$$

i) $\frac{\partial f}{\partial x_1} = 2 + 6x_1 + 4x_2 \Rightarrow \frac{\partial^2 f}{\partial x_1^2} = 6$

ii) $\frac{\partial f}{\partial x_2} = 6x_2 + 4x_1 \Rightarrow \frac{\partial^2 f}{\partial x_2^2} = 6$

iii) $\frac{\partial f}{\partial x_1 \partial x_2} = 4$

$\therefore H = \begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix}$

$\det(H - \lambda I) = 0$

$\begin{vmatrix} 6-\lambda & 4 \\ 4 & 6-\lambda \end{vmatrix} = (6-\lambda)^2 - 16 = 0$

$6 - \lambda = \pm 4 \Rightarrow \lambda_1 = 2, \lambda = 10$

λ_1, λ_2 둘다 양수이므로 행렬 H는 positive definite이고,

따라서, 함수 $f(x_1, x_2)$는 convex이다.

(2) $f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 16 & 23 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \pi^2$

여기에서 Hessian Matrix는 $H = \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix}$ 이고

$\det \begin{bmatrix} 3-\lambda & 3 \\ 1 & 3-\lambda \end{bmatrix} = 0$

$(3-\lambda)^2 - 3 = 0$, $3 - \lambda = \pm\sqrt{3}$

$\therefore \lambda = 3 \pm \sqrt{3}$

$(\lambda_1 = 3 + \sqrt{3}, \lambda_2 = 3 - \sqrt{3})$

λ_1, λ_2 둘 다 양수이기 때문에 행렬 H는 positive definite이고, 따라서 Convex이다.

(3) $f(x, y) = 3(x^2 + y^2) + 4xy + 5x + 6y + 7$

Hessian Matrix H는 다음과 같이 적을 수 있다.

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

i) $\frac{\partial^2 f}{\partial x^2} = 6$ ii) $\frac{\partial^2 f}{\partial y^2} = 6$ iii) $\frac{\partial^2 f}{\partial x \partial y} = 4$

따라서, Hessian Matrix

$H = \begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix}$ 이고,

문제 (1) 일 때와 Hessian Matrix가 동일하므로,

고유값 λ_1, λ_2 또한 양수일 것이므로

$f(x, y)$는 convex하다.

1-2. Obtain the optimal solution to the above problems using CVX.

rs₩brian₩OneDrive₩바탕 화면₩최적화개론 project1(최윤석)₩Project1-(1)₩Project1_1_2.m

```
%%Function 1

%Define the Function 1 as an anonymous function of a 2D vector x
fun1 = @(x) 1 + 2*x(1) + 3*(x(1)^2 + x(2)^2) + 4*x(1)*x(2);
x0 = [0; 0]; % Initial guess
%Use fminunc to find the minimum of the Function 1 starting from x0
[x_opt1, fval1] = fminunc(fun1, x0);

%% Function2
% 초기값 설정
x0 = [0; 0];  % 초기 추정값

% 비대칭 Q 정의
Q = [3 3; 1 3];  % 원래 비대칭 행렬
b = [16; 23];    % 선형항 벡터

% Q 대칭화
Q_sym = (Q + Q') / 2;  % Q를 대칭 행렬로 변환

% 목적 함수 정의 (이제는 대칭 Q 사용)
fun2 = @(x) x' * Q_sym * x + b' * x + pi^2;

% fminunc 옵션 설정
options = optimoptions('fminunc', 'Algorithm', 'quasi-newton', ...
                       'Display', 'off', ...
                       'OptimalityTolerance', 1e-12);

% 수치적 최적화 수행 (fminunc)
[x_opt, fval_num] = fminunc(fun2, x0, options);

% 해석적 해 계산
x_analytical = -0.5 * inv(Q_sym) * b;
fval_analytical = x_analytical' * Q_sym * x_analytical + b' * x_analytical + pi^2;


%% Function 3

% Define the third function with quadratic, cross-product, and linear terms
fun3 = @(x) 3*(x(1)^2 + x(2)^2) + 4*x(1)*x(2) + 5*x(1) + 6*x(2) + 7;

% Find the minimum of the third function using fminunc
[x_opt3, fval3] = fminunc(fun3, x0);

%% Function1 결과 출력
disp('Function 1:')
disp(['Optimal Solution: x1* = ', num2str(x_opt1(1)), ', x2* = ', num2str(x_opt1(2))])
disp(['Optimal Cost: f* = ', num2str(fval1)])

%% Function2 결과 출력
disp('Function 2 Optimization with Symmetric Q:')
disp(['Numerical Solution (fminunc): x* = [' num2str(x_opt(1)) ', ' num2str(x_opt(2)) ']'])
disp(['Numerical Optimal Cost: f* = ', num2str(fval_num)])
disp(' ')

disp(['Analytical Solution:        x* = [' num2str(x_analytical(1)) ', ' num2str(x_analytical(2)) ']'])
disp(['Analytical Optimal Cost:    f* = ', num2str(fval_analytical)])


%% Function3 결과 도출
disp('Function 3:')
disp(['Optimal Solution: x1* = ', num2str(x_opt3(1)), ', x2* = ', num2str(x_opt3(2))])
disp(['Optimal Cost: f* = ', num2str(fval3)])
```

Code Result

```
▸ 바탕 화면 ▸ 최적화개론 project1(최윤석) ▸ Project1-(1)
명령 창
>> Project1_1_2

국소 최솟값을 찾았습니다.

기울기의 크기가 최적성 허용오차의
값보다 작기 때문에 최적화가 완료되었습니다.

<중지 기준 세부 정보>

국소 최솟값을 찾았습니다.

기울기의 크기가 최적성 허용오차의
값보다 작기 때문에 최적화가 완료되었습니다.

<중지 기준 세부 정보>
Function 1:
Optimal Solution: x1* = -0.6, x2* = 0.4
Optimal Cost: f* = 0.4
Function 2 Optimization with Symmetric Q:
Numerical Solution (fminunc): x* = [-0.2, -3.7]
Numerical Optimal Cost: f* = -34.2804

Analytical Solution:          x* = [-0.2, -3.7]
Analytical Optimal Cost:      f* = -34.2804
Function 3:
Optimal Solution: x1* = -0.3, x2* = -0.8
Optimal Cost: f* = 3.85
```

1-3.Obtain the optimal solution and optimal cost for each function by your hand.

$f(x) = \frac{1}{2} x^T H x + c^T x + d$

(for a strictly convex quadratic function)

여기에 대한 최적해 (optimal solution) x^* 은

다음과 같은 조건을 만족한다.

$Hx^* + C = 0$

Function 1 : $H = \begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix}$, $C = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$

$$\begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix} \begin{bmatrix} x_1^* \\ x_2^* \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 0$$

$$\begin{bmatrix} 6x_1^* + 4x_2^* \\ 4x_1^* + 6x_2^* \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 0$$

$6x_1^* + 4x_2^* = -2$

$4x_1^* + 6x_2^* = 0$

$\therefore x_1^* = -0.6, \quad x_2^* = 0.4$

$\therefore f(x_1^*, x_2^*) = 1 - 1.2 + 3(0.36 + 0.16) - 0.96$

$= 0.4$

Function 2 : $f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 16 & 23 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \pi^2$

$= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 16 & 23 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \pi^2$

즉, $f(x) = x^T Q x + b^T x + \pi^2$ 형태이다.

여기서 $Q = \begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix}$ $b = \begin{bmatrix} 16 \\ 23 \end{bmatrix}$

⇒ 하지만, 행렬 Q는 비대칭이므로, 해석적으로 풀기 위해서는 Q를 대칭화 해야 함.

* 대칭화한 행렬 Q_{sym} :

$$Q_{sym} = \frac{Q + Q^T}{2} = \frac{1}{2}\left(\begin{bmatrix} 3 & 3 \\ 1 & 3 \end{bmatrix} + \begin{bmatrix} 3 & 1 \\ 3 & 3 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

이차 목적함수가 다음과 같은 형태로 정의되었다면

$$f(x) = x^T Q_{sym} x + b^T x + \pi^2$$

⇒ 최적해는 $2 Q_{sym} x^* + b = 0 \rightarrow x^* = -\frac{1}{2} Q_{sym}^{-1} b$

$$Q_{sym}^{-1} = \frac{1}{\det(Q)} \begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix} = \frac{1}{3 \cdot 3 - 2 \cdot 2} \begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix}$$

$$= \frac{1}{5} \begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix}$$

$$x^* = -\frac{1}{2} \frac{1}{5} \begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} 16 \\ 23 \end{bmatrix} = -\frac{1}{10} \begin{bmatrix} (3)(16) - (2)(23) \\ (-2)(16) + 3(23) \end{bmatrix}$$

$$= -\frac{1}{10} \begin{bmatrix} 2 \\ 37 \end{bmatrix} = \begin{bmatrix} -0.2 \\ -3.7 \end{bmatrix}$$

※ $\pi^2 = 9.8696$

이제 x^*를 위의 목적함수에 대입해주면

$$(x^*)^T Q_{sym} x^* = \begin{bmatrix} -0.2 & -3.7 \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} -0.2 \\ -3.7 \end{bmatrix}$$

$$= 44.15$$

$$b^T x^* = \begin{bmatrix} 16 & 23 \end{bmatrix} \begin{bmatrix} -0.2 & -3.7 \end{bmatrix} = -3.2 - 85.1 = -88.3$$

$$\therefore f(x^*) = 44.15 - 88.3 + 9.8696 \approx -34.2804$$

Function 3 : $f(x, y) = 3(x^2 + y^2) + 4xy + 5x + 6y + 7$

$$\Rightarrow \frac{1}{2} \begin{bmatrix} x^* & y^* \end{bmatrix} \begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix} \begin{bmatrix} x^* \\ y^* \end{bmatrix} + \begin{bmatrix} p & q \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x^* \\ y^* \end{bmatrix} + r?$$

$$\begin{bmatrix} 6 & 4 \\ 4 & 6 \end{bmatrix} \begin{bmatrix} x^* \\ y^* \end{bmatrix} = \begin{bmatrix} -5 \\ -6 \end{bmatrix}$$

$$6x^* + 4y^* = -5$$
$$4x^* + 6y^* = -6 \quad \Rightarrow x^* = -0.3 \quad y^* = -0.8$$

$$\therefore f(x^*, y^*) = 3x^{*2} + 3y^{*2} + 4x^* y^* + 5x^* + 6y^* + 7$$

$$= 3 \times (-0.3)^2 + 3 \times (-0.8)^2 + 4 \times (-0.3)(-0.8) + 5 \times (-0.3) + 6 \times (-0.8) + 7$$

$$= 3.85$$

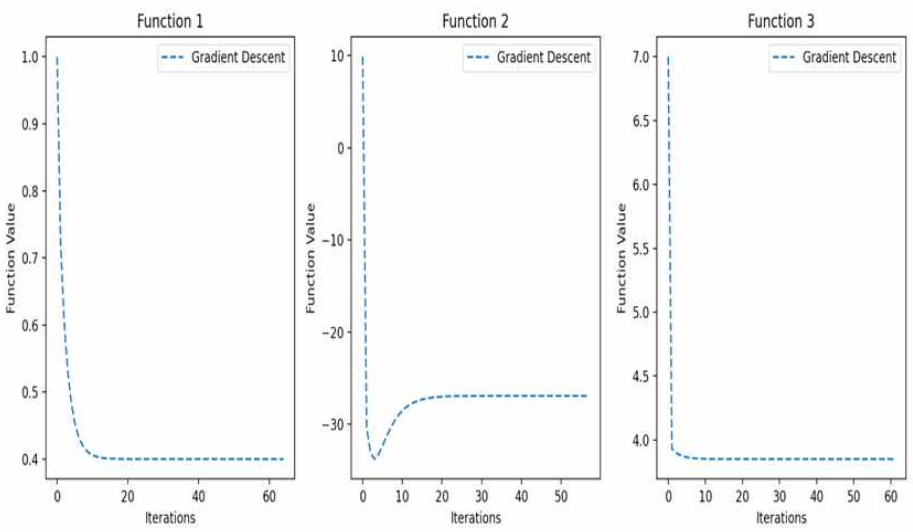
1-4. Use Python to design the gradient descent algorithm and find the optimal solution and optimal value. Also, obtain the convergence plot.

PJ1_Problem1 (4).py

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(1) > PJ1_Problem1 (4).py > gradient_descent > to

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import inv

# Define the three functions and their gradients
def function1(x):
    x1, x2 = x
    return 1 + 2*x1 + 3*(x1**2 + x2**2) + 4*x1*x2

def gradient1(x):
    x1, x2 = x
    df_dx1 = 2 + 6*x1 + 4*x2
    df_dx2 = 6*x2 + 4*x1
    return np.array([df_dx1, df_dx2])

def function2(x):
    Q = np.array([[3, 3], [1, 3]])
    b = np.array([16, 23])
    return x.T @ Q @ x + b.T @ x + np.pi**2

def gradient2(x):
    Q = np.array([[3, 3], [1, 3]])
    b = np.array([16, 23])
    return 2 * Q @ x + b

def function3(x):
    x1, x2 = x
    return 3*(x1**2 + x2**2) + 4*x1*x2 + 5*x1 + 6*x2 + 7

def gradient3(x):
    x1, x2 = x
    df_dx1 = 6*x1 + 4*x2 + 5
    df_dx2 = 6*x2 + 4*x1 + 6
    return np.array([df_dx1, df_dx2])
```

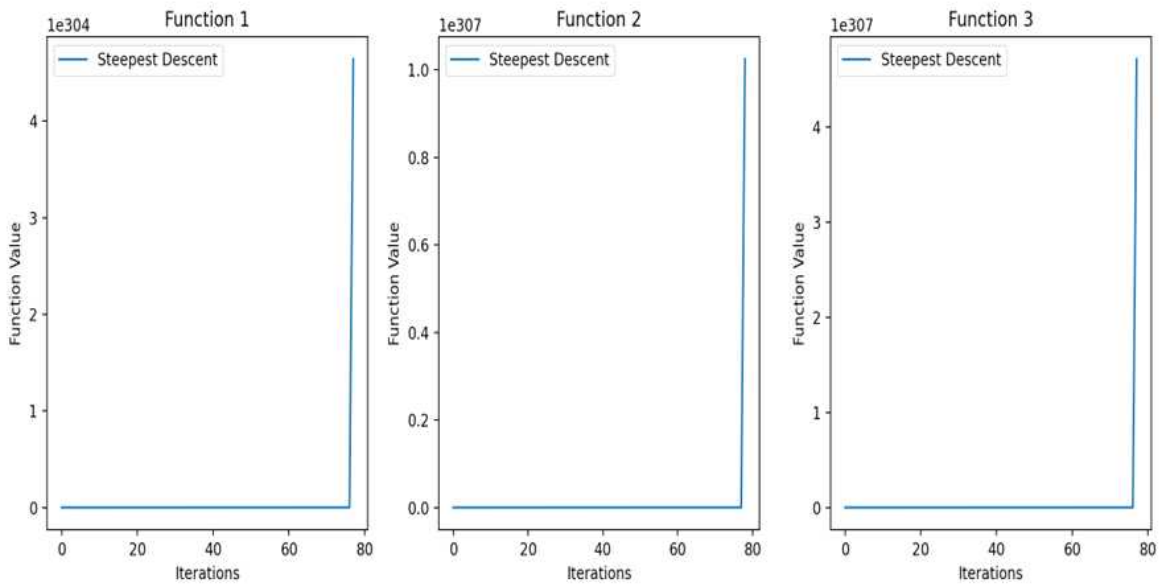
```python
# Gradient Descent Algorithm
def gradient_descent(func, grad, x0, alpha=0.1, tol=1e-6, max_iter=1000):
    x = np.array(x0, dtype=float)
    values = []
    for _ in range(max_iter):
        values.append(func(x))
        grad_x = grad(x)
        if np.linalg.norm(grad_x) < tol:
            break
        x -= alpha * grad_x
    return x, values

# Run gradient descent for each function
functions = [(function1, gradient1), (function2, gradient2), (function3, gradient3)]
x0_list = [[0, 0], [0, 0], [0, 0]]
names = ["Function 1", "Function 2", "Function 3"]

plt.figure(figsize=(12, 5))
for i, (func, grad) in enumerate(functions):
    x0 = x0_list[i]
    gd_x, gd_vals = gradient_descent(func, grad, x0)

    # Print the optimal solution and optimal function value
    optimal_value = func(gd_x)
    print(f"{names[i]}:")
    print(f"  Optimal Solution: x* = {gd_x}")
    print(f"  Optimal Value: f(x*) = {optimal_value}\n")

    plt.subplot(1, 3, i+1)
    plt.plot(gd_vals, label='Gradient Descent', linestyle='dashed')
    plt.xlabel('Iterations')
    plt.ylabel('Function Value')
    plt.title(names[i])
    plt.legend()

plt.tight_layout()
plt.show()
```



1-5. Use Python to design the steepest (optimal step size) gradient descent and find the optimal solution and optimal value. Also, obtain the convergence plot.

```
# 각각의 함수 정의 및 각각의 함수의 Gradient 정의
def function1(x):
    x1, x2 = x
    return 1 + 2*x1 + 3*(x1**2 + x2**2) + 4*x1*x2

def gradient1(x):
    x1, x2 = x
    return np.array([6*x1 + 4*x2 + 2, 6*x2 + 4*x1])

def function2(x):
    Q = np.array([[3, 3], [1, 3]])
    b = np.array([16, 23])
    return x.T @ Q @ x + b.T @ x + np.pi**2

def gradient2(x):
    Q = np.array([[3, 3], [1, 3]])
    b = np.array([16, 23])
    return 2 * Q @ x + b

def function3(x):
    x1, x2 = x
    return 3*(x1**2 + x2**2) + 4*x1*x2 + 5*x1 + 6*x2 + 7

def gradient3(x):
    x1, x2 = x
    return np.array([6*x1 + 4*x2 + 5, 6*x2 + 4*x1 + 6])

def hessian1():
    return np.array([[6, 4], [4, 6]])


def hessian2():
    return 2 * np.array([[3, 3], [1, 3]])

def hessian3():
    return np.array([[6, 4], [4, 6]])
```

```python
# Steepest Descent 알고리즘
def steepest_descent(func, grad, hessian, x0, tol=1e-6, max_iter=1000):
    x = np.array(x0, dtype=float)
    values = []
    for _ in range(max_iter):
        values.append(func(x))
        grad_x = grad(x)
        if np.linalg.norm(grad_x) < tol:
            break
        H_inv = inv(hessian())
        alpha = (grad_x.T @ H_inv @ grad_x) / (grad_x.T @ H_inv @ H_inv @ grad_x)
        x -= alpha * grad_x
    return x, values

# 초기값 및 함수 선택
functions = [(function1, gradient1, hessian1),(function2, gradient2, hessian2), (function3, gradient3, hessian3)]
x0_list = [[0, 0],[0, 0], [0, 0]]
names = ["Function 1","Function 2", "Function 3"]

plt.figure(figsize=(12, 5))
for i, (func, grad, hessian) in enumerate(functions):
    x0 = x0_list[i]
    sd_x, sd_vals = steepest_descent(func, grad, hessian, x0)

    # 최적해 및 최적함수값 출력
    optimal_value = func(sd_x)
    print(f"{names[i]}:")
    print(f"  Optimal Solution: x* = {sd_x}")
    print(f"  Optimal Value: f(x*) = {optimal_value}\n")


    plt.subplot(1, 3, i+1)
    plt.plot(sd_vals, label='Steepest Descent', linestyle='solid')
    plt.xlabel('Iterations')
    plt.ylabel('Function Value')
    plt.title(names[i])
    plt.legend()
```

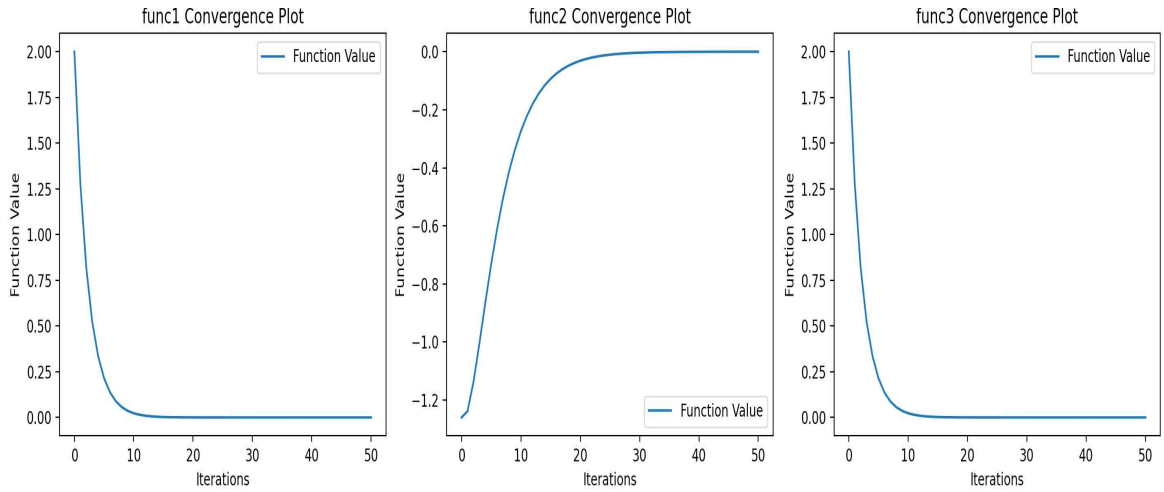


1-6. **Let x* be the optimal solution obtained by 3. Let be the optimal solution obtained by 4. Let be the optimal solution obtained by 5. . Plot the convergence of ||x*-x*(Gradient)|| , ||x*-x*(Steepest Gradient)||.**

```python
#각각의 함수 정의

def func1(x1, x2):
    return (x1 + 0.6)**2 + (x2 - 0.4)**2

def grad_func1(x1, x2):
    return np.array([2 * (x1 + 0.6), 2 * (x2 - 0.4)])

def func2(x1, x2):
    return (x1 +0.2)+(x2 +3.7)**2

def grad_func2(x1, x2):
    return np.array([2 * (x1 +0.2), 2 * (x2 +3.7)])

def func3(x, y):
    return (x + 0.3)**2 + (y + 0.8)**2

def grad_func3(x, y):
    return np.array([2 * (x + 0.3), 2 * (y + 0.8)])

def steepest_gradient(grad):
    norm = np.linalg.norm(grad)
    return -grad / norm if norm != 0 else np.zeros_like(grad)

# 최적해
opt_solutions = {
    'func1': np.array([-0.6, 0.4]),
    'func2': np.array([-0.2 -3.7]),
    'func3': np.array([-0.3, -0.8])
}
```

```python
# Plot 설정
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

for i, (name, opt_x) in enumerate(opt_solutions.items()):
    # Gradient와 Steepest Gradient 차이 계산
    x_vals, grad_vals, steepest_vals = [], [], []
    x = opt_x + np.array([1.0, -1.0])  # 최적해 근처의 초기값 설정

    for _ in range(num_iterations):
        grad = globals()[f'grad_{name}'](*x)
        steepest_grad = steepest_gradient(grad)

        # 값 저장
        x_vals.append(np.linalg.norm(x - opt_x))
        grad_vals.append(np.linalg.norm(x - grad))
        steepest_vals.append(np.linalg.norm(x - steepest_grad))

        # Gradient Descent 업데이트
        x -= learning_rate * grad

      # Store final optimal solution
    optimal_solution = x
    optimal_value = globals()[name](*optimal_solution)

    # Print the optimal results
    print(f"{name}:")
    print(f"  Optimal Solution: x* = {optimal_solution}")
    print(f"  Optimal Value: f(x*) = {optimal_value}\n")
```

```
    # 첫 번째 줄: 절댓값 차이 플로팅
    axes[0, i].plot(range(num_iterations), np.abs(np.array(x_vals) - np.array(grad_vals)), label='|opt_x - grad|')
    axes[0, i].plot(range(num_iterations), np.abs(np.array(x_vals) - np.array(steepest_vals)), label='|opt_x - steepest_grad|', linestyle='dashed')
    axes[0, i].set_title(f'{name} Gradient Difference')
    axes[0, i].legend()
    axes[0, i].set_xlabel('Iterations')
    axes[0, i].set_ylabel('Difference')

    # 두 번째 줄: 함수 값 수렴 플로팅
    func_vals = [globals()[name](*opt_x + np.array([1.0, -1.0]))]
    x = opt_x + np.array([1.0, -1.0])
    for _ in range(num_iterations):
        grad = globals()[f'grad_{name}'](*x)
        x -= learning_rate * grad
        func_vals.append(globals()[name](*x))

    axes[1, i].plot(range(num_iterations + 1), func_vals, label='Function Value')
    axes[1, i].set_title(f'{name} Convergence Plot')
    axes[1, i].legend()
    axes[1, i].set_xlabel('Iterations')
    axes[1, i].set_ylabel('Function Value')

plt.tight_layout()
plt.show()
```



1-7. Discuss the convergence speed in the convergence plots in 5 and 6.

For all functions (Function 1, 2, 3), the function value explodes exponentially as the iteration increases. In other words, the function value does not decrease but rather continues to increase, failing to converge. Therefore, in the Steepest Descent Method Plot of 5, Steepest Descent diverged because it was not properly tuned, and the algorithm did not converge.

In all three functions(Plot 6), the function value steadily decreases with the iteration and converges stably. Functions 1 and 3 almost converge after about 10 to 15 iterations. Compared to Function 1 and Function 3, Function 2 steadily increases as the iteration increases and converges stably.

In particular, the slow convergence of Function 2 eventually reaches stability, securing

the reliability of the algorithm.
Therefore, the algorithm in Plot 6 is relatively more stable than that in Plot 5.

Problem 2

Consider the quadratic programming

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x$$
$$\nabla f(x) = Qx - b$$
$$Q = Q^\top > 0, \; b \in \mathbb{R}^n$$

Use “randn” in numpy to generate a 1000 x 1000 matrix A. Then obtain Q by

$$Q = AA^\top + \rho I$$

Here, rho is any positive number such that Q is positive definite.
Use “randn” in numpy to generate 1000 x 1 vector b

2-1. Show that the functions are convex.

#2-1 Show that the function is convex.

(sol) 함수를 두 번 편미분하여 Hessian 산출
⇒ Hessian이 Positive Definite 일경우 함수는 Convex

$\nabla f = Qx - b$
$H = \nabla^2 f = Q = AA^T + \varrho I$

ϱ : any positive number that makes Q positive-definite

$v^T \cdot H \cdot v = v^T \cdot A \cdot A^T \cdot v + \varrho v^T \cdot v$
$= \|A \cdot v\|^2 + \varrho \|v\|^2 \geq 0$

∴ H is positive-definite and f is Convex.

2-2. Obtain the optimal solution and optimal cost using CVX.

```python
import numpy as np
import cvxpy as cp
from math import sqrt
import matplotlib.pyplot as plt

A = np.random.randn(1000,1000) #random 1000*1000 matrix
while np.linalg.matrix_rank(A) != 1000: #recreate A if not full rank until A is full rank
    A = np.random.randn(1000,1000) #recreate A if not full rank

rho = np.random.uniform(low = 0.0, high = 1000.0) #random positive scalar for Q to be positive definite

Q = np.dot(A, A.T) + rho * np.identity(1000) #create matrix Q
b = np.random.randn(1000) #random 1000 dim vector b
initial_x = np.random.randn(1000) #random initial input for x

def function2(x: np.ndarray): #function to calc function result
    result = 0.5 * np.dot(x.T, np.dot(Q,x)) - np.dot(b.T, x)
    gradient = np.dot(Q,x) - b
    return result, gradient
```

```python
#obtain optimal solution utilizing CVXPY library
# https://www.cvxpy.org/examples/
x_cvx = cp.Variable(1000) #delcare x_cvx as variable
function_cvx = 0.5 * ((x_cvx.T) @ Q @ x_cvx) - (b.T @ x_cvx) #declare problem
prob = cp.Problem(cp.Minimize(function_cvx)) #define problem
prob.solve() #solve
print("CVXPY Result:\n", "Opt Value: ", prob.value) #print cost
#print("\nX: ",x_cvx.value)
```

2-3. Use Python to design the gradient descent algorithm and find the optimal solution and optimal value. Also, obtain the convergence plot.

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(2) > PJ1_Problem2.py > ...

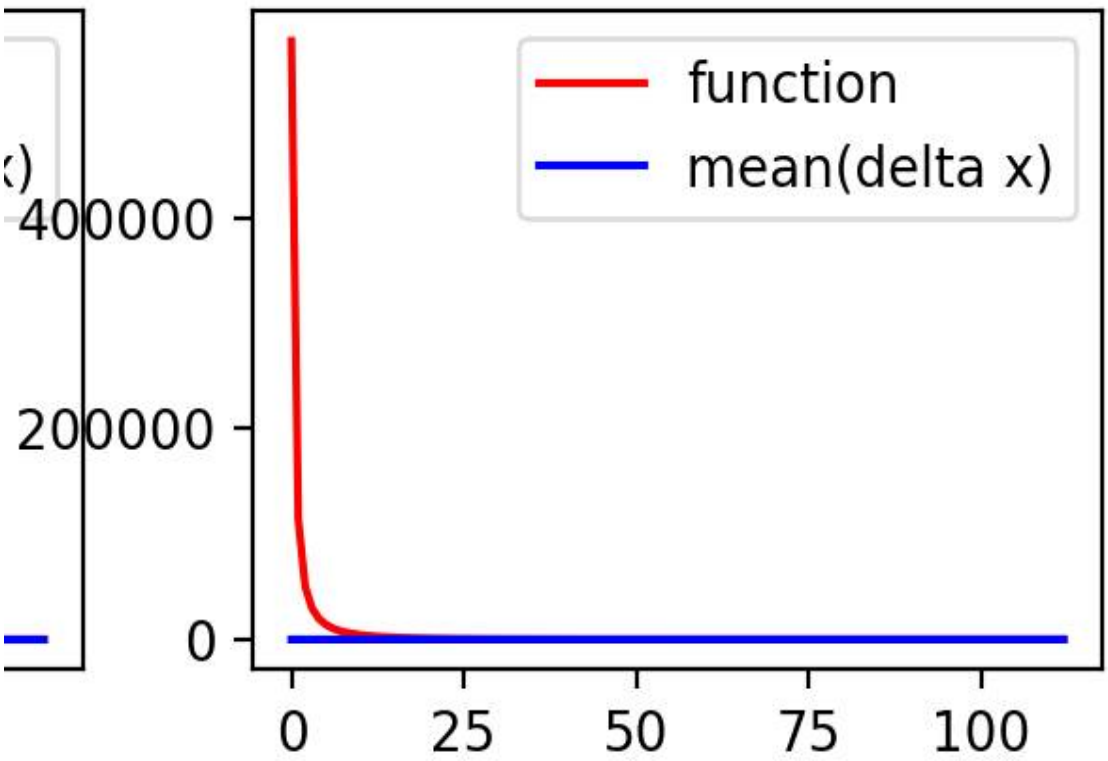
```python
def function2(x: np.ndarray): #function to calc function result
    result = 0.5 * np.dot(x.T, np.dot(Q,x)) - np.dot(b.T, x)
    gradient = np.dot(Q,x) - b
    return result, gradient
```

```python
#initialize x,y matrices
x_gradient = np.zeros(shape = (1000, max_iteration), dtype = np.double) #initialize x
y_gradient = np.zeros(shape = (max_iteration,), dtype = np.double) #initialize y
x_gradient[:,0] = initial_x
y_gradient[0], gradient = function2(x_gradient[:,0]) #initial y

for n in range(1, max_iteration):       #run gradient descent algorithm
    x_gradient[:,n] = x_gradient[:,n-1] - stepsize * gradient   #calculate new x vextor
    y_gradient[n], gradient = function2(x_gradient[:,n])    #calculate new y
    if abs(y_gradient[n]-y_gradient[n-1]) < epsilon:    #stopping criterion
        iteration_end_GD = n+1              #note when iteration stopped
        break                           #stop the loop
else:
    iteration_end_GD = max_iteration    #if the convergence condition is not satisfied

x_gradient = x_gradient[:, :iteration_end_GD]   #cut x according to # of iteration
y_gradient = y_gradient[:iteration_end_GD]      #cut y according to # of iteration


#show result
print("GP- Number of Iteration: ", iteration_end_GD) #print how much iteration it took
print("result: ", y_gradient[iteration_end_GD-1])
delta_gradient = np.mean(np.absolute(x_gradient - np.repeat(np.array(x_cvx.value).reshape(1000,1), iteration_end_GD, axis =1)),axis = 0) #obtain abs(x_k -
#show convergence plot

iteration_gradient = range(0,iteration_end_GD) #array for X-axis plot
plt.subplot(2,2,1)#plot top left
plt.plot(iteration_gradient, y_gradient, 'r',label = 'function')#convergence plot of function
plt.plot(iteration_gradient, delta_gradient, 'b', label = 'mean(delta x)')#convergence plot of abs(x-x*)
plt.legend()#plot legend
```

2-4. Use Python to design the steepest (optimal step size) gradient descent and find the optimal solution and optimal value. Also, obtain the convergence plot.

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(2) > PJ1_Problem2.py > ...

```python
#steepest GD
#initialize x,y matrices
x_sgd = np.zeros(shape = (1000, max_iteration), dtype = np.double) #initialize x
y_sgd = np.zeros(shape = (max_iteration,), dtype = np.double) #initialize y
x_sgd[:,0] = initial_x
y_sgd[0], gradient = function2(x_sgd[:,0]) #initial y

for n in range(1, max_iteration):      #run gradient_descent alogorithm
    stepsize_sgd = np.dot(gradient.T, gradient)/np.dot(gradient.T,np.dot(Q,gradient)) #calc steepest GD stepsize
    x_sgd[:,n] = x_sgd[:,n-1] - stepsize_sgd * gradient     #calculate new x vector
    y_sgd[n], gradient = function2(x_sgd[:,n])            #calculate new y
    if abs(y_sgd[n]-y_sgd[n-1]) < epsilon:     #stopping criterion
        iteration_end_sgd = n+1               #note when iteration stopped
        break                                 #stop the loop
else:
    iteration_end_sgd = max_iteration       #if the convergence condition is not satisfied

x_sgd = x_sgd[:, :iteration_end_sgd]        #cut x according to # of iteration
y_sgd = y_sgd[:iteration_end_sgd]           #cut y according to # of iteration


#show result
print("Steepest GD- Number of Iteration: ", iteration_end_sgd) #print how much iteration it took
print("result: ", y_sgd[iteration_end_sgd-1])

delta_sgd = np.mean(np.absolute(x_sgd - np.repeat(np.array(x_cvx.value).reshape(1000,1), iteration_end_sgd, axis =1)),axis = 0) #obtain abs(x_k -x*)
#show convergence plot
iteration_sgd = range(0,iteration_end_sgd) #array for X-axis plot
plt.subplot(2,2,2) #plot top right
plt.plot(iteration_sgd, y_sgd, 'r',label = 'function')#convergence plot of function
plt.plot(iteration_sgd, delta_sgd, 'b', label = 'mean(delta x)')#convergence plot of abs(x-x*)
plt.legend()#plot legend
```

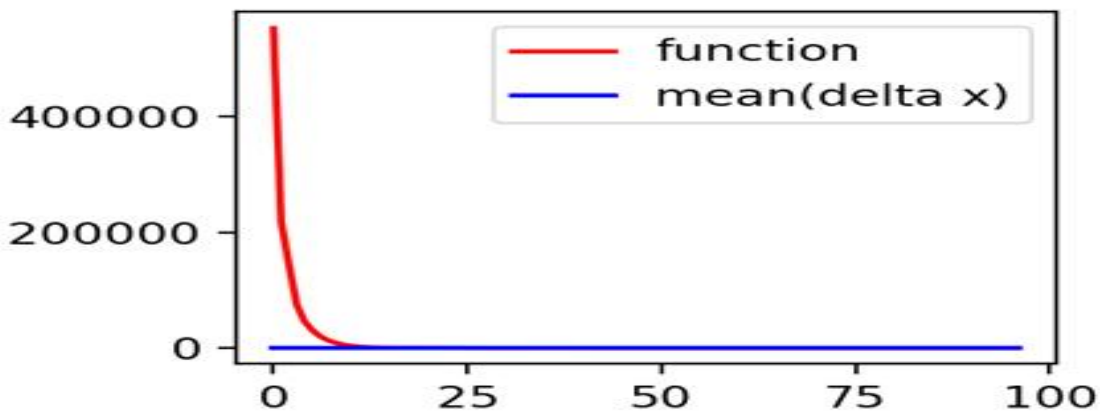


2-5. Use Python to design the Nesterov-2 algorithm and find the optimal solution and optimal value. Also, obtain the convergence plot.

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(2) > PJ1_Problem2.py > ...

```python
#Nesterov-2 Algorithm
stepsize = 0.0005 #step size
alpha = np.array([0.5, 0]) #initial alpha value = 0.5 #initial beta value

#initialize x,y matrices
x_nesterov = np.zeros(shape = (1000, max_iteration), dtype = np.double) #initialize x matrix
f = np.zeros(shape = (max_iteration,), dtype = np.double) #vector for tracking function result
y = np.zeros(shape = (1000,2), dtype = np.double) #1000*2 for y_k and y_(k+1)
x_nesterov[:,0] = np.random.randn(1000) #randomize initial input for x
f[0], gradient = function2(x_nesterov[:,0]) #initial function value

#run Nesterov-2 algorithm
for n in range(1, max_iteration):
    k = (n-1)%2    #calculation for alternating value of k
    k2 = n%2        #calculation for alternating value of k+1(for saving memory)
    stepsize = np.dot(gradient.T, gradient)/np.dot(gradient.T,np.dot(Q,gradient))   #calculate steepest step size
    alpha[k2] = 0.5*(sqrt(alpha[k]**4 + 4*(alpha[k]**2))-alpha[k]**2)    #calculate alpha_k+1
    beta = alpha[k]*(1-alpha[k])/(alpha[k]**2 + alpha[k2])               #calculate beta_k
    y[:,k2] = x_nesterov[:,n-1] - stepsize * gradient                    #calculate y_(k+1)
    x_nesterov[:,n] = y[:,k2] + beta * (y[:,k2]-y[:,k])                  #calculate x_(k+1)
    f[n], gradient = function2(x_nesterov[:,n])                          #calculate function cost and gradient
    if abs(f[n]-f[n-1]) < epsilon:                                       #stopping criterion
        iteration_end_N2 = n+1                                           #note when iteration stopped
        break                                                            #stop the loop
else:
    iteration_end_N2 = max_iteration                                      #if the convergence condition is not satisfied

x_nesterov = x_nesterov[:,:iteration_end_N2] #discard unused memory
f = f[:iteration_end_N2] #discard unused memory
```

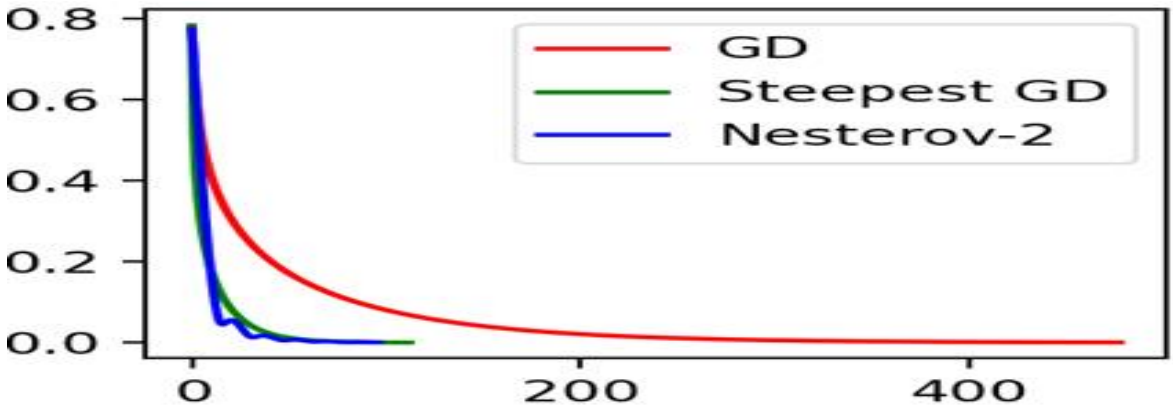
```python
#show result
print("Nesterov-2 Algorithm -Number of Iteration: ", iteration_end_N2) #print number of iteration
print("result: ", f[iteration_end_N2 -1]) #print result cost

delta_nesterov = np.mean(np.absolute(x_nesterov - np.repeat(np.array(x_cvx.value).reshape(1000,1), iteration_end_N2, axis =1)),axis = 0) #obtain abs(x_k ->
#show convergence plot
iteration_N2 = range(0,iteration_end_N2) #array for X-axis plot
plt.subplot(2,2,3) #plot bottom left
plt.plot(iteration_N2, f, 'r',label = 'function')#convergence plot of function
plt.plot(iteration_N2, delta_nesterov, 'b', label = 'mean(delta x)')#convergence plot of abs(x-x*)
plt.legend() #plot legend

#compare results between algorithms
plt.subplot(2,2,4) #plot bottom
plt.plot(iteration_gradient, delta_gradient,'r', label = 'GD') #plot mean error of gradient descent
plt.plot(iteration_sgd, delta_sgd, 'g', label = 'Steepest GD') #plot mean error of steepest gradient descent
plt.plot(iteration_N2, delta_nesterov, 'b', label = 'Nesterov-2') #plot mean error of steepest gradient descent

plt.legend() #plot legend
plt.show() #show plot
```



2-6. **Let x* be the optimal solution obtained by 2. Let x*(Gradient) be the optimal solution obtained by 3. Let x*(Steepest Gradient) be the optimal solution obtained by 4. Let x*(Nestrov-2) be the optimal solution obtained by Nestrov-2 in 5. Plot the convergence of ||x*-x*(Gradient)|| , ||x*-x*(Steepest Gradient)||, ||x*-x*(Nestrov-2)||.**

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(2) > PJ1_Problem2.py > ...

```
140  #compare results between algorithms
141  plt.subplot(2,2,4) #plot bottom
142  plt.plot(iteration_gradient, delta_gradient,'r', label = 'GD') #plot mean error of gradient descent
143  plt.plot(iteration_sgd, delta_sgd, 'g', label = 'Steepest GD') #plot mean error of steepest gradient descent
144  plt.plot(iteration_N2, delta_nesterov, 'b', label = 'Nesterov-2') #plot mean error of steepest gradient descent
145
146  plt.legend() #plot legend
147  plt.show() #show plot
```



2-7. Discuss the convergence speed in the convergence plots in 6.

**Convergence Speed
Nesterov−2>SteepestGradientDescent>>GradientDescent

The Nesterov−2 algorithm, which has the largest reduction in iterations, is the fastest, and Steepest GD, which computes the optimal step size for QP, has a large speed advantage over regular Gradient Descent.

Problem 3

3-1. Find the optimal solution using the gradient descent method with the initial condition that you choose

$$f(x_1, x_2) = 100(x_2 - {x_1}^2)^2 + (1 - x_1)^2$$

To implement the Gradient Descent Method, we need to find the partial differentiation for each variable.

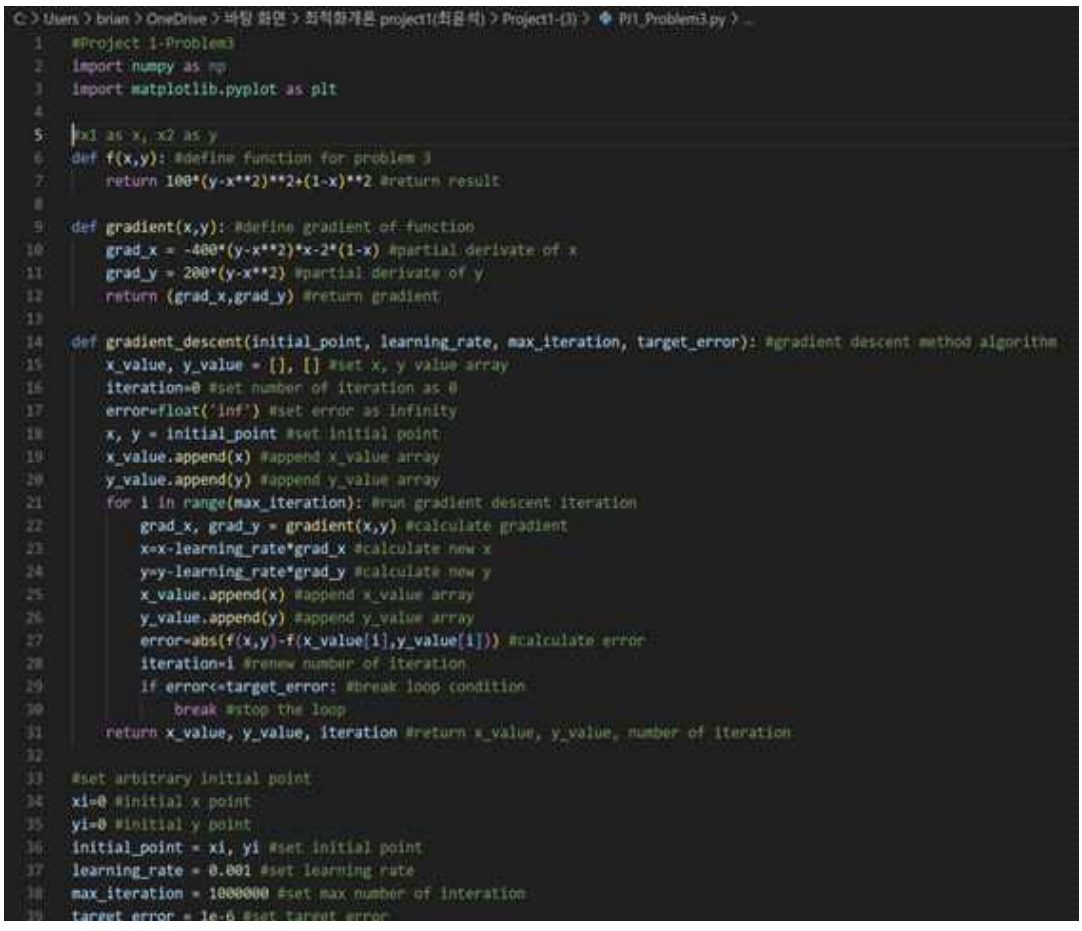
$$\frac{\partial}{\partial x_1} f(x_1, x_2) = -400(x_2 - {x_1}^2)^2 - 2(1 - x_1)$$

$$\frac{\partial}{\partial x_2} f(x_1, x_2) = 200(x_2 - {x_1}^2)$$

Let's arbitrarily set the initial value of the function f to be optimized as follows,

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k), \; x_0 = x$$

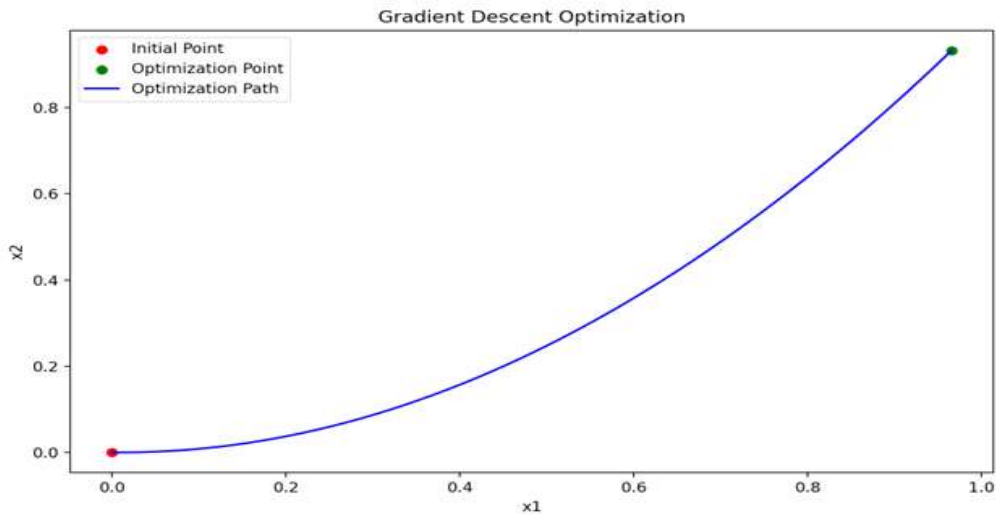
Repeat the Gradient Descent Method until the target error is reached, and output the values including the number of iterations and the optimal solution. Plot the initial values, values including the optimal solution, and the path to find the optimal solution.



```
C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(3) > P1_Problem3.py > ...
#Project 1-Problem3
import numpy as np
import matplotlib.pyplot as plt

#x1 as x, x2 as y
def f(x,y): #define function for problem 3
    return 100*(y-x**2)**2+(1-x)**2 #return result

def gradient(x,y): #define gradient of function
    grad_x = -400*(y-x**2)*x-2*(1-x) #partial derivate of x
    grad_y = 200*(y-x**2) #partial derivate of y
    return (grad_x,grad_y) #return gradient

def gradient_descent(initial_point, learning_rate, max_iteration, target_error): #gradient descent method algorithm
    x_value, y_value = [], [] #set x, y value array
    iteration=0 #set number of iteration as 0
    error=float('inf') #set error as infinity
    x, y = initial_point #set initial point
    x_value.append(x) #append x_value array
    y_value.append(y) #append y_value array
    for i in range(max_iteration): #run gradient descent iteration
        grad_x, grad_y = gradient(x,y) #calculate gradient
        x=x-learning_rate*grad_x #calculate new x
        y=y-learning_rate*grad_y #calculate new y
        x_value.append(x) #append x_value array
        y_value.append(y) #append y_value array
        error=abs(f(x,y)-f(x_value[i],y_value[i])) #calculate error
        iteration=i #renew number of iteration
        if error<=target_error: #break loop condition
            break #stop the loop
    return x_value, y_value, iteration #return x_value, y_value, number of iteration

#set arbitrary initial point
xi=0 #initial x point
yi=0 #initial y point
initial_point = xi, yi #set initial point
learning_rate = 0.001 #set learning rate
max_iteration = 1000000 #set max number of interation
target_error = 1e-6 #set target error
```

```
41  x_value, y_value, iteration = gradient_descent(initial_point, learning_rate, max_iteration, target_error) #run gradient descent method algorithm
42
43  print('iteration =',iteration) #print number of interation
44  print('initial_point(x1,x2) = (', xi, ',', yi,')') #print initial point
45  print('opt_point(x1,x2) = (', x_value[len(x_value)-1],',', y_value[len(y_value)-1],')') #print optimization point
46  print('opt_value =', f(x_value[len(x_value)-1], y_value[len(y_value)-1])) #print optimization value
47
48  plt.figure(figsize=(10,6)) #set plot size
49  plt.scatter(xi, yi, c='r', label='Initial Point') #plot initial point
50  plt.scatter(x_value[len(x_value)-1], y_value[len(y_value)-1], c='g', label='Optimization Point') #plot optimization point
51  plt.plot(x_value, y_value, c='b', label='Optimization Path') #plot optimization path
52  plt.title('Gradient Descent Optimization') #set title
53  plt.xlabel('x1') #set x label
54  plt.ylabel('x2') #set y label
55  plt.legend() #plot legend
56  plt.show() #show plot
```

When the initial point was set to (0, 0), the following number of iterations and the same value as the optimal solution were obtained.

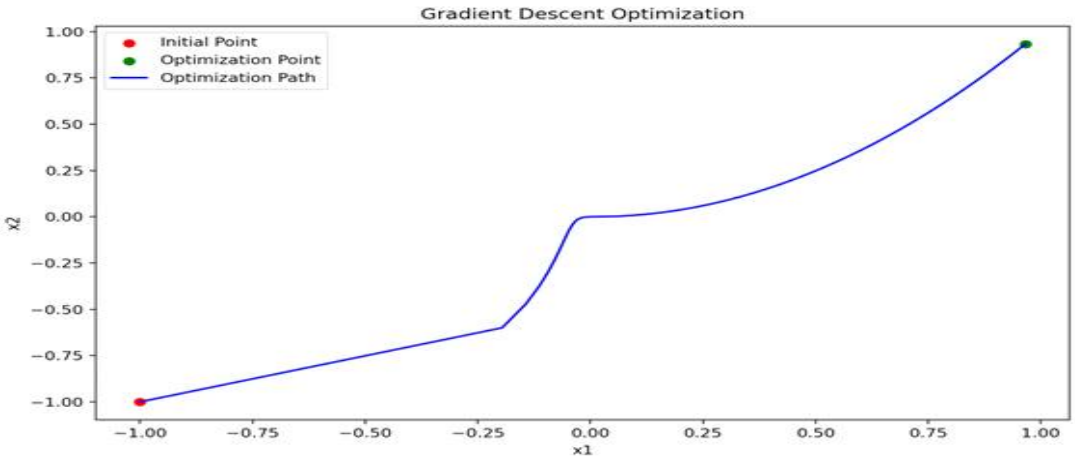



```
iteration = 5570
initial_point(x1,x2) = ( 0 , 0 )
opt_point(x1,x2) = ( 0.9656285040585441 , 0.932297997695398 )
opt_value = 0.0011833712344146508
```

3-2. Gradient descent method with different initial condition.

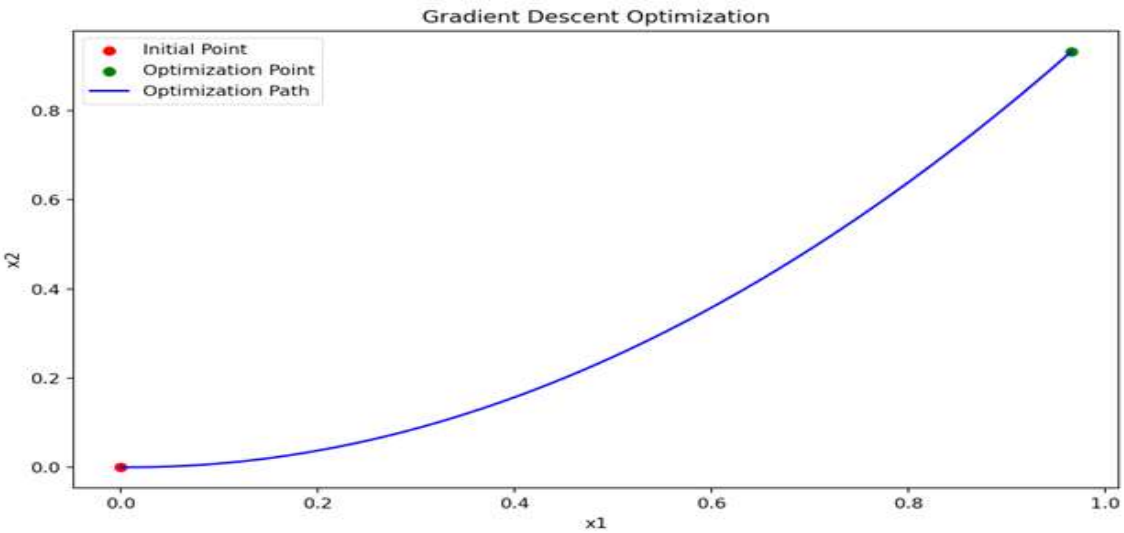
Initial Point: (-1, -1)

```
iteration = 5601
initial_point(x1,x2) = ( -1 , -1 )
opt_point(x1,x2) = ( 0.9656354754469227 , 0.9323114903520062 )
opt_value = 0.0011828912327907317
```



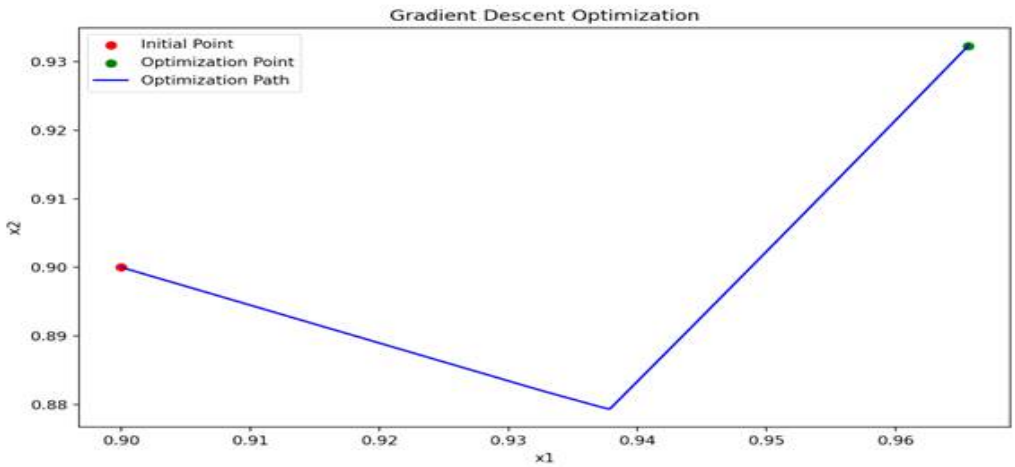
Initial Point: (0, 0)

```
iteration = 5570
initial_point(x1,x2) = ( 0 , 0 )
opt_point(x1,x2) = ( 0.9656285040585441 , 0.932297997695398 )
opt_value = 0.0011833712344146508
```



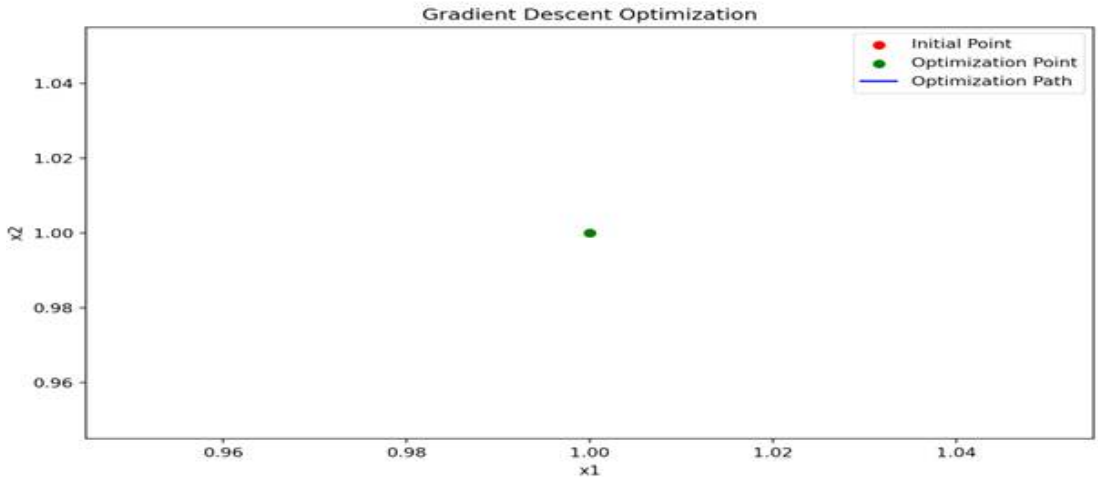
Initial Point: (0.9, 0.9)

```
iteration = 1376
initial_point(x1,x2) = ( 0.9 , 0.9 )
opt_point(x1,x2) = ( 0.9656265905584123 , 0.9322942942606495 )
opt_value = 0.0011835030018413316
```



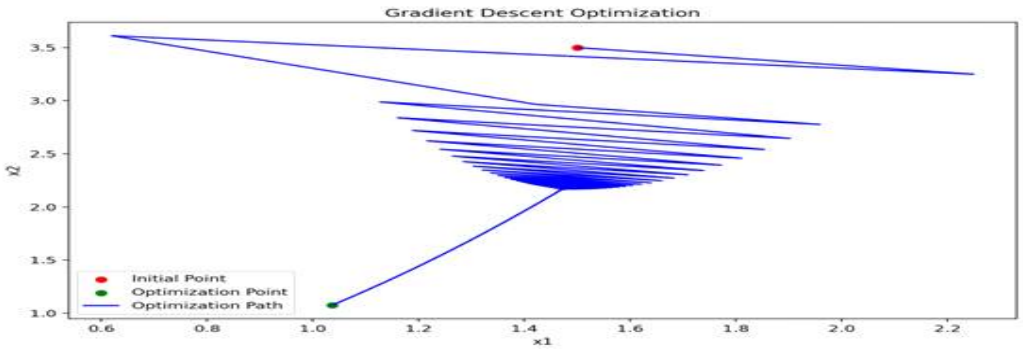
Initial Point: (1.0, 1.0)

```
iteration = 0
initial_point(x1,x2) = ( 1.0 , 1.0 )
opt_point(x1,x2) = ( 1.0 , 1.0 )
opt_value = 0.0
```



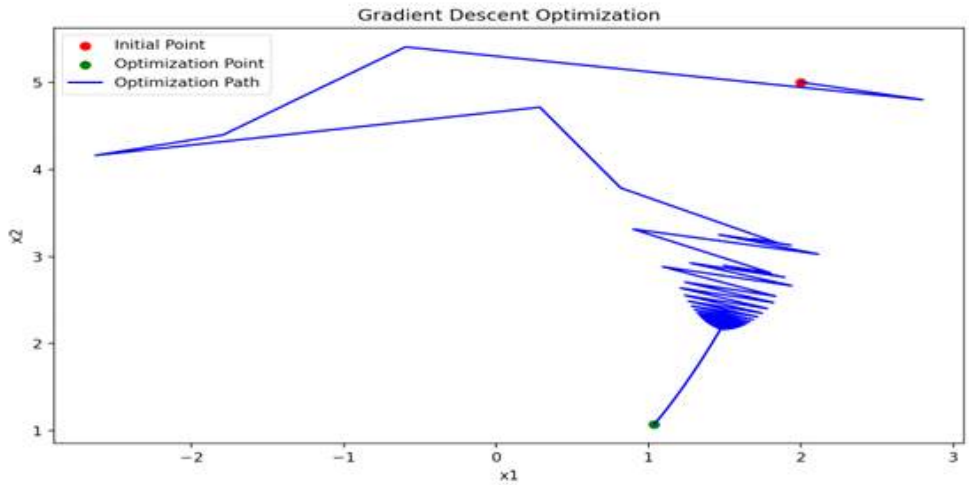
Initial Point: (1.5, 3.5)

```
iteration = 8458
initial_point(x1,x2) = ( 1.5 , 3.5 )
opt_point(x1,x2) = ( 1.0363721905978869 , 1.0742097128068049 )
opt_value = 0.001324963892804972
```



****Initial Point: (2.0, 5.0)****

```
iteration = 8469
initial_point(x1,x2) = ( 2.0 , 5.0 )
opt_point(x1,x2) = ( 1.03637696354611 , 1.0742196242079496 )
opt_value = 0.0013253116412893329
```



The closer the initial point is to the solution (1,1), the fewer the number of iterations, and the faster the convergence speed. If the initial point is set too far from the solution, convergence does not occur. Therefore, in order to apply the gradient design method to find the optimal solution, the optimal solution must be known, and the surrounding area must be set as the initial point.

Problem 4

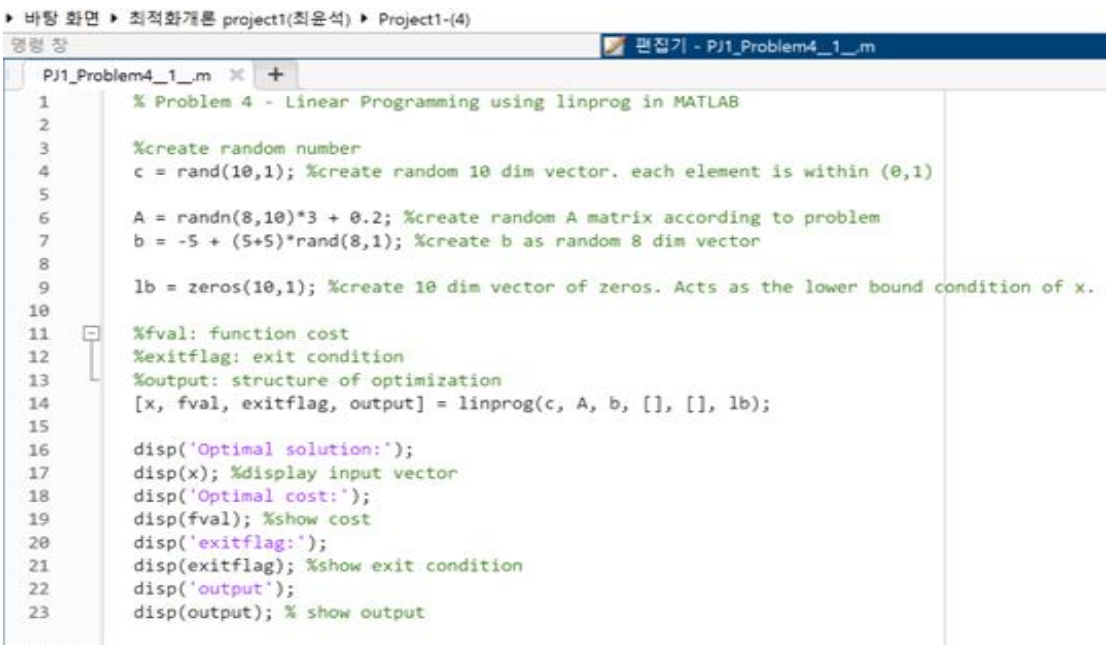
$$\min_{x \in \mathbb{R}^n} c^\top x$$
$$\text{subject to } Ax \leq b$$
$$x_i \geq 0, \quad i = 1, \ldots, n$$

$$\text{note that}$$
$$c \in \mathbb{R}^n, \quad A \in \mathbb{R}^{p \times n} \ (p \leq n), \quad b \in \mathbb{R}^p$$
$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

• Use MATLAB or Python to generate random c, A, b data (e.g. for MATLAB)

• c = rand(10,1), A = randn(8,10)*3+0.2, b = -5 + (5+5)*rand(8,1)

• Use linprog in MATLAB or Python to find the optimal cost and optimal solution with the random data given above.



(Example Solution)

```
>> PJ1_Problem4__1__

최적해를 구했습니다.

Optimal solution:
           0
           0
           0
      0.1093
      0.6549
           0
           0
      0.0563
           0
           0

Optimal cost:
      0.3949

exitflag:
      1

output
           iterations: 3
            algorithm: 'dual-simplex-highs'
      constrviolation: 8.3267e-17
              message: '최적해를 구했습니다.'
        firstorderopt: 1.1102e-16
```

• Use MATLAB or Python to generate random c, A, b data (e.g. for MATLAB)

• c = rand(100,1), A = randn(55,100)*5-1.2, b = -1+(1+1)*rand(55,1)

• Use linprog in MATLAB or Python to find the optimal cost and optimal solution with the random data given above.

바탕 화면 ▸ 최적화개론 project1(최윤석) ▸ Project1-(4)

명령 창

편집기 - PJ1_Problem4__2__.m

PJ1_Problem4__2__.m

```
1   %create random number
2   c = rand(100,1); %create random 100 dim vector. each element is within (0,1)
3   A = randn(55,100)*5 - 1.2; %create random A matrix according to problem
4   b = -1 + (1+1)*rand(55,1); %create b as random 8 dim vector
5
6   lb = zeros(100,1); %create 10 dim vector of zeros. Acts as the lower bound condition of x.
7
8   %fval: function cost
9   %exitflag: exit condition
10  %output: structure of optimization
11  [x, fval, exitflag, output] = linprog(c, A, b, [], [], lb);
12
13  disp('Optimal solution:');
14  disp(x); %display input vector
15  disp('Optimal cost:');
16  disp(fval); %show cost
17  disp('exitflag:');
18  disp(exitflag); %show exit condition
19  disp('output');
20  disp(output); % show output
```

(Example Solution)

Optimal Value: 0.0353, 0.0130, 0.0208, 0.0061, 0.0006, 0.0185, 0.0187, 0.0028, 0.0576, 0.0025, 0.0795, 0.0000, 0.0394, 0.0063, 0.0130, 0.0168, 0.0100, 0.0203, 0.0300

(List of all except 0)

```
Optimal cost:
    0.0630

exitflag:
     1

output
          iterations: 35
           algorithm: 'dual-simplex-highs'
     constrviolation: 2.7439e-15
             message: '최적해를 구했습니다.'
       firstorderopt: 7.7716e-16
```

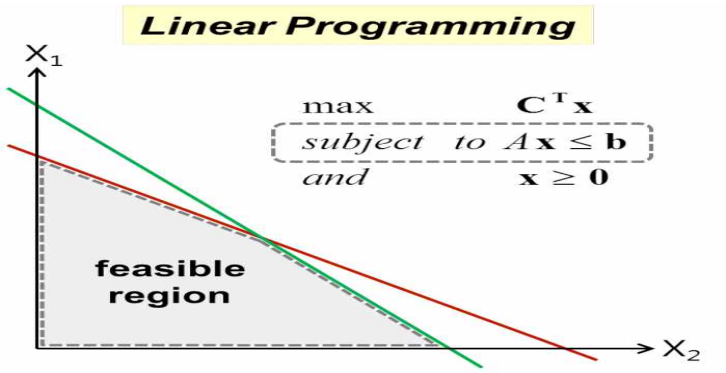
- **Discuss the location of the optimal solution in terms of the constraints**

$$Ax \leq b$$
$$x_i \geq 0, \; i = 1, \ldots, n$$

The objective function is given by $c^T x$, which is a linear function.The constraints are $Ax \leq b$ and $x_i \geq 0$, which are also linear. The variable x is an n-dimensional vector, and the goal of this problem is to minimize the objective function.

The key point here is that both the objective function and the constraints are linear, which means we can apply the properties of linear programming. In a linear programming problem, the optimal solution always lies on the boundary or at a vertex of the feasible region.This is because a linear objective function can be seen as a plane that tilts in a certain direction, and the minimum value occurs at the lowest point where this plane intersects the feasible region — typically at a vertex of the region.

*Soruce: https://gaussian37.github.io/



As shown in the figure above, the feasible region can be viewed as a polygon defined by multiple linear constraints.
If we move the level set (contour line) of the linear objective function until it touches the feasible region, the point where it makes its final contact at the lowest position becomes the optimal solution. This point may be a vertex or an entire boundary face of the polygon. Therefore, the optimal solution of this problem lies on the boundary of the feasible region defined by the constraints —and most commonly, at a vertex. This is a fundamental property of linear programming.
Thanks to this characteristic, we don't need to search the entire region to find the optimal solution —
it's sufficient to examine just the vertices, which significantly reduces the complexity.

Problem 5

$$\min_{x \in \mathbb{R}^n} f(x)$$

$$\text{where } f(x) = \|Ax - b\|_Q^2 = (Ax - b)^\top Q(Ax - b)$$

$$A \in \mathbb{R}^{m \times n}$$

$$b \in \mathbb{R}^m$$

$$Q = Q^\top \in \mathbb{R}^{m \times n} : \text{ positive definite matrix}$$

5-1. Show that the problem is convex.

$f(x) = (Ax - b)^T Q(Ax - b)$ 에서 convexity임을 보이려면 Hessian Matrix $\nabla^2 f(x)$가 positive semi definite인지 확인

⟶ 여기서 $f(x)$는 quadratic function이고, quadratic function의 Hessian은 $A^T QA$ 형태로 나타남.

⟶ 만약 Q가 positive definite이면 $A^T QA$ 도 항상 positive semi definite이므로, 이는 convex 함수가 됨.

근거: 모든 Affine Set는 항상 Convex Set임(단, 역은 성립하지 않음.)

(Reference**) Proof of 'All Affine Set is Convex Set'**

Since an Affine Set can have a linear combination that holds for all real coefficients, it includes cases where λ is greater than or equal to 0 and less than or equal to 1. That is, if λ = θ∈ [0,1], then θx1+(1−θ)x2∈S. As can be seen from the equation above, an Affine Set always

contains a convex combination, so it satisfies the definition of a Convex Set.

5-2. Find the optimal solution to this problem by your hand.

$$f(x) = (Ax - b)^T Q(Ax - b)$$

A is an m×n matrix, B is an m×1 matrix, and Q is a symmetric postivedefinite matrix of m×n. Since Q must be a square matrix as a positive definite matrix, assume that the size of Q is m×m and proceed with solving the problem.

1) 목적함수 전개

$f(x) = (Ax-b)^T Q(Ax-b)$

$= x^T A^T QAx - x^T A^T Qb - b^T QAx + b^T Qb$

2) 목적함수의 Gradient 구하기

f(x)의 최솟값을 구하기 위해, Gradient $\nabla f(x)$를 구한후 0으로 만드는 값을 찾아야한다.

$\nabla f(x) = \nabla (x^T A^T QAx - x^T A^T Qb - b^T QAx + b^T Qb)$

$\nabla_x (x^T A^T QAx) = 2A^T QAx$ ($\because A^T QA$는 대칭행렬)

$\nabla_x (x^T A^T Qb) = A^T Qb$ ($\because A^T QA$는 대칭행렬)

$\therefore \nabla f(x) = 2A^T QAx - A^T Qb - (A^T Qb)$

$= 2A^T QAx - 2A^T Qb$

3) Gradient 0으로 설정하기

$\nabla f(x) = 2A^T QAx - 2A^T Qb = 0$

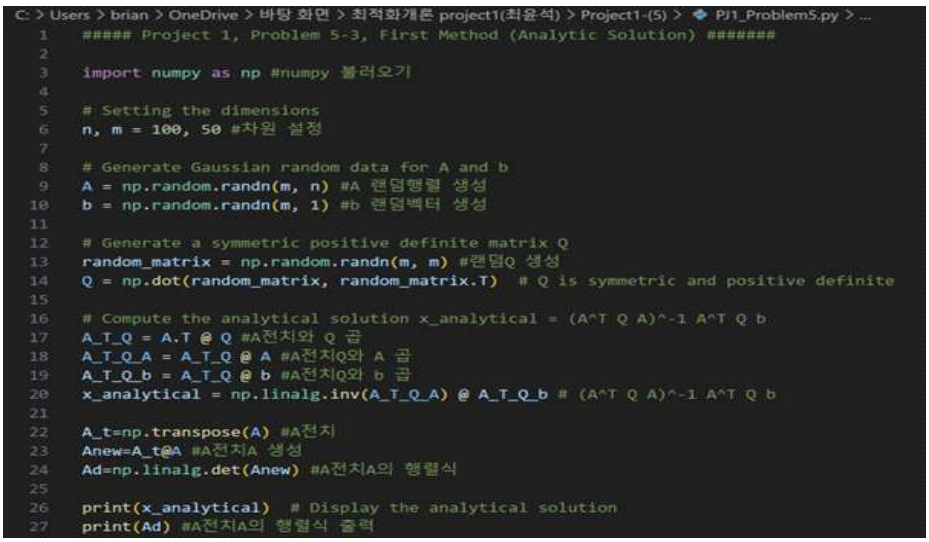
$\Rightarrow A^T QAx = A^T Qb$

4) x에 대해 풀이

$A^T QA$가 가역행렬 (invertible matrix)인 선형 최소 자승 문제 (Least Square Problem)와 같으므로 해는 다음과 같다.

$\therefore x = (A^T QA)^{-1} A^T Qb$

5-3. Let n=100, m=50, use Python to generate the Gaussian random data of A, b, Q. Then find the optimal solution via the gradient descent method and the solution of 2. Also use CVX to find the optimal solution.
Analytic Solution



```
C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(5) > PJ1_Problem5.py > ...
1   ##### Project 1, Problem 5-3, First Method (Analytic Solution) #######
2
3   import numpy as np #numpy 불러오기
4
5   # Setting the dimensions
6   n, m = 100, 50 #차원 설정
7
8   # Generate Gaussian random data for A and b
9   A = np.random.randn(m, n) #A 랜덤행렬 생성
10  b = np.random.randn(m, 1) #b 랜덤벡터 생성
11
12  # Generate a symmetric positive definite matrix Q
13  random_matrix = np.random.randn(m, m) #랜덤Q 생성
14  Q = np.dot(random_matrix, random_matrix.T)  # Q is symmetric and positive definite
15
16  # Compute the analytical solution x_analytical = (A^T Q A)^-1 A^T Q b
17  A_T_Q = A.T @ Q #A전치와 Q 곱
18  A_T_Q_A = A_T_Q @ A #A전치Q와 A 곱
19  A_T_Q_b = A_T_Q @ b #A전치Q와 b 곱
20  x_analytical = np.linalg.inv(A_T_Q_A) @ A_T_Q_b # (A^T Q A)^-1 A^T Q b
21
22  A_t=np.transpose(A) #A전치
23  Anew=A_t@A #A전치A 생성
24  Ad=np.linalg.det(Anew) #A전치A의 행렬식
25
26  print(x_analytical)  # Display the analytical solution
27  print(Ad) #A전치A의 행렬식 출력
```

(Analytic Solution에 의한 출력 결과 예시)

[[-114. , 74. , 62. , 188. , -40. , 117.875, -28. , -14. , -158.5 , 20. ,
-100. , 98. , -73. , 48. , 40.5 , 106. , -12. , 166.25 , -24. , -91.5 ,
61. , -29.5 , -48. , -168. , 116. , -126. , 94. , -49. , 48. , -121. ,
196. , 80. , 170.75 , 4. , 138.5 , 218. , 31.75 , -9. , -15.5 , -41. ,
-104. , 52. , -231. , -13. , -6. , 36. , 64. , 164. , -61.234, -98. ,
24. , -106. , 46. , -12. , -28.25 , 46. , -26. , 28.5 , 11. , -30. ,
-37. , -32. , 80. , 65. , 38.375, -2. , -16. , 2. , 31. , -4. ,
2. , 10. , -22. , -14.5 , -10. , 13.5 , 21. , -3.5 , -2.75 , -2. ,
-50. , -9. , -5.5 , 8. , 14. , 17.75 , 13.25 , -4. , 2. , 4. ,
2. , -11.625, 1. , -20. , -1. , 0. , -3.5 , -2. , 0. , -3.625]]

First, by implementing the analytical solution in Python code, we can see that the solution is properly obtained. Obviously, since only the matrix elements were directly substituted into the equation, the result was normally obtained.

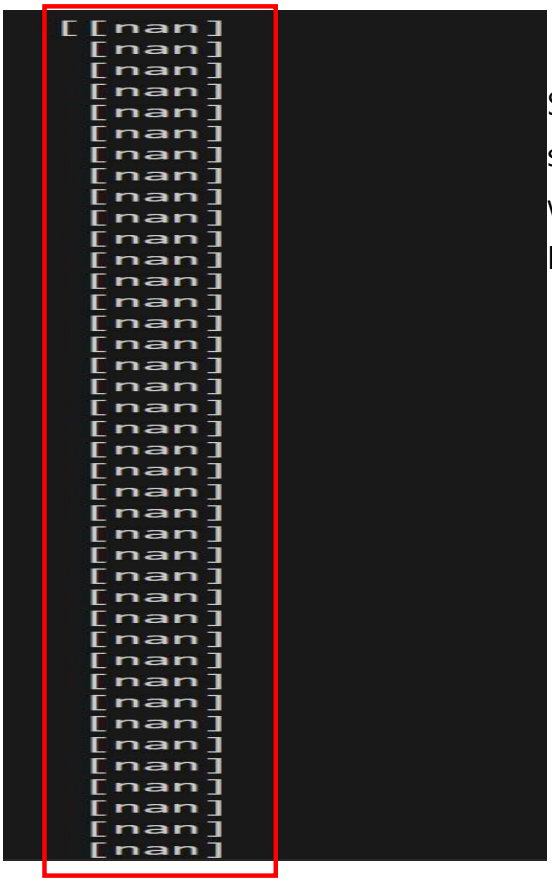
Gradient Descent Method

C: > Users > brian > OneDrive > 바탕 화면 > 최적화개론 project1(최윤석) > Project1-(5) > PJ1_Problem5.py > ...

```python
###### Project 1, Problem 5-3, Second Method (Gradient Descent) #######

import numpy as np

def gradient_descent(A, b, Q, x_init, learning_rate, max_iter, tol): #경사하강법 시작
    x = x_init #초깃값 선언
    f_prev = float('inf') #'무한'을 나타내는 값 만들어주기

    for i in range(max_iter): #최대 반복횟수까지 연산
        # Compute the gradient
        grad = 2 * A.T @ Q @ (A @ x - b) #그라디언트 선언

        # Update x
        x = x - learning_rate * grad # x 업데이트

        # Compute the value of the function
        f_x = ((A @ x - b).T @ Q @ (A @ x - b)).item() #목적함수 선언

        # Check for convergence
        if abs(f_prev - f_x) < tol: #만약 오차허용범위보다 값들의 차이가 작으면
            break #계산 중지
        f_prev = f_x #최종값 반환

    return x, i+1  # Return the solution and the number of iterations
```

```python
# Initial guess for x
x_init = np.random.randn(n, 1) #x0 초깃값 선언

# Parameters for gradient descent
learning_rate = 0.001 #학습률은 0.001로 설정
max_iter = 1000 # 최대반복횟수는 1000회
tol = 1e-6 #오차 허용 범위는 1e-6

# Perform gradient descent
x_gd, iterations = gradient_descent(A, b, Q, x_init, learning_rate, max_iter, tol) #결과 계산함수

x_gd, iterations  # Display the gradient descent solution and number of iterations

A_t=np.transpose(A) # A 전치 생성
Anew=A_t@A # A전치A 행렬 만들기
Ad=np.linalg.det(Anew) # 행렬식 계산

print(x_gd, iterations)  # Display the analytical solution
print(Ad) # 행렬식 선언
```

*Print Result



Since n is in a Rank Deficient state smaller than m, a NaN error is output when using the Gradient Descent Method.

5-4. Let n=50, m=100, use Python to generate the Gaussian random data of A, b, Q. Then find the optimal solution via the gradient descent method and the solution of 2. Also use CVX to find the optimal solution.

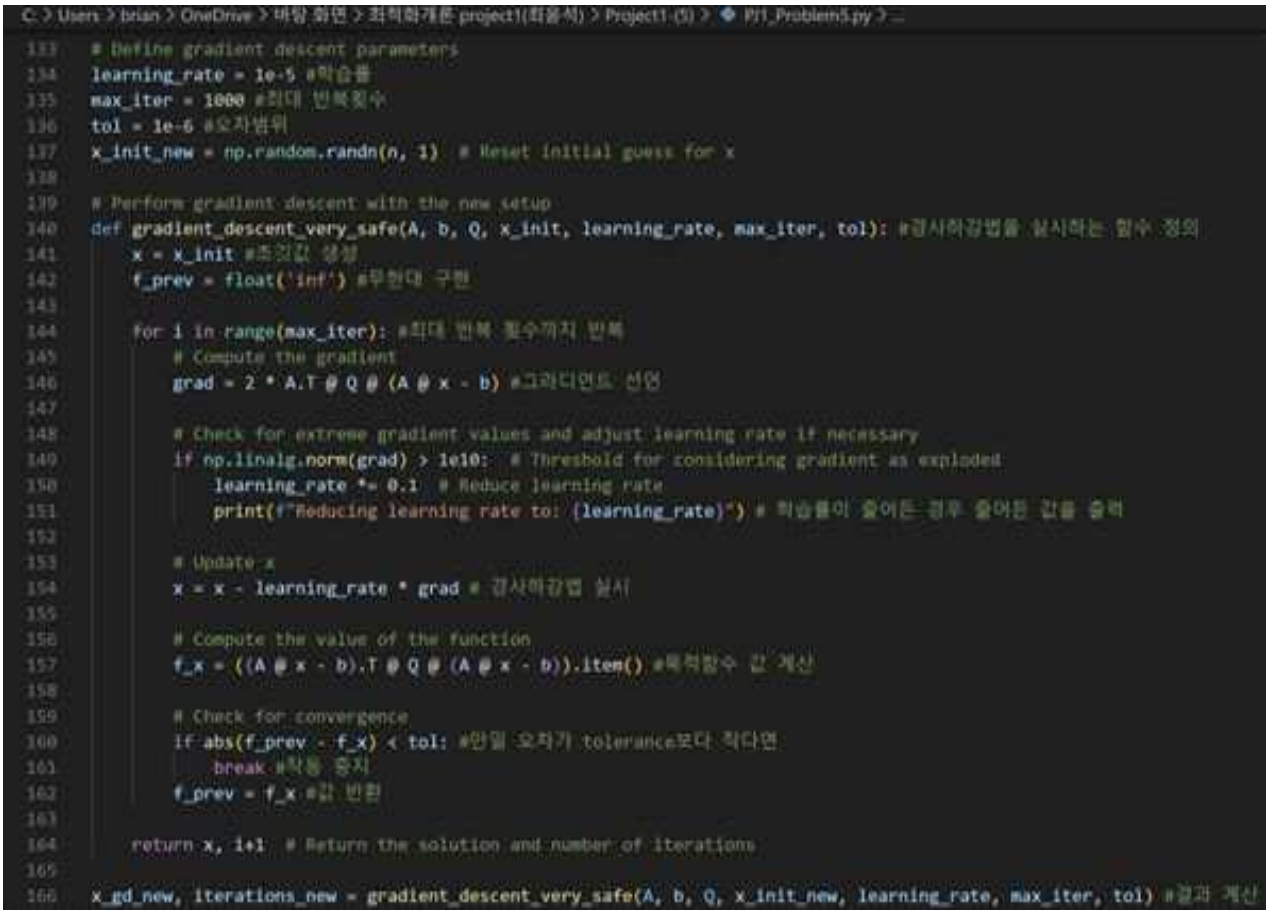
Analytic Solution

```python
import numpy as np # numpy 라이브러리 호출

# Redefine dimensions for the new problem setup
n, m = 50, 100 #차원 설정

# Generate Gaussian random data for A and b with new dimensions
A = np.random.randn(m, n) #랜덤 A 생성
b = np.random.randn(m, 1) #랜덤 B 생성

# Generate a symmetric positive definite matrix Q (m x m)
random_matrix = np.random.randn(m, m) # 랜덤 Q 생성
Q = np.dot(random_matrix, random_matrix.T)  # Q remains symmetric and positive definite

# Compute the analytical solution x_analytical = (A^T Q A)^-1 A^T Q b
A_T_Q = A.T @ Q # A^T Q 곱 계산
A_T_Q_A = A_T_Q @ A # A^TQA 계산
A_T_Q_b = A_T_Q @ b # A^TQb 계산
x_analytical_new = np.linalg.inv(A_T_Q_A) @ A_T_Q_b #해석해 출력
```

(Example of Results by Analytic Solution)

[[-0.2145]
[0.1663]
[0.0791]
...
[-0.0173]
[-0.1397]]

*Gradient Descent Method *



```
# Define gradient descent parameters
learning_rate = 1e-5 #학습률
max_iter = 1000 #최대 반복횟수
tol = 1e-6 #오차범위
x_init_new = np.random.randn(n, 1)  # Reset initial guess for x

# Perform gradient descent with the new setup
def gradient_descent_very_safe(A, b, Q, x_init, learning_rate, max_iter, tol): #경사하강법을 실시하는 함수 정의
    x = x_init #초깃값 설정
    f_prev = float('inf') #무한대 구현

    for i in range(max_iter): #최대 반복 횟수까지 반복
        # Compute the gradient
        grad = 2 * A.T @ Q @ (A @ x - b) #그라디언트 선언

        # Check for extreme gradient values and adjust learning rate if necessary
        if np.linalg.norm(grad) > 1e10:  # Threshold for considering gradient as exploded
            learning_rate *= 0.1  # Reduce learning rate
            print(f"Reducing learning rate to: {learning_rate}") # 학습률이 줄어든 경우 줄어든 값을 출력

        # Update x
        x = x - learning_rate * grad # 경사하강법 실시

        # Compute the value of the function
        f_x = ((A @ x - b).T @ Q @ (A @ x - b)).item() #목적함수 값 계산

        # Check for convergence
        if abs(f_prev - f_x) < tol: #만일 오차가 tolerance보다 작다면
            break #학습 중지
        f_prev = f_x #값 반환

    return x, i+1  # Return the solution and number of iterations

x_gd_new, iterations_new = gradient_descent_very_safe(A, b, Q, x_init_new, learning_rate, max_iter, tol) #결과 계산
```

(Example of Results by Gradient Descent Method)

[[-0.2145]
[0.1663]
[0.0791]
...
[-0.0173]
[-0.1397]]

Unlike 5-3, you can see that both the Analytic Solution and the Gradient Descent Method produce the same solution.

5-5. Discuss the solutions of 3 and 4. Provide your discussion in terms of the matrix inverse and rank condition.

	m=50, n=100	m=100, n=50
$\det(A^T A)$ 값		

In the case of 5-3, the A matrix is 50x100 in size, and there is a high risk of it becoming a Rank-Deficient Matrix. In other words, the Condition Number is large and there is a high risk of it becoming Non-invertible. On the other hand, in the case of 5-4, the A matrix is 100 X 50 in size, and there is a high possibility of it becoming Full Rank. Specifically, if you look at the size of , you can see that 5-4 is larger than 5-3.

Therefore, the Gradient Descent Method works better in 5-4 than in 5-3.

(Note) Some of the code comments have been deleted, but the comments are attached to the code file.